Lonely Chess

A new programming language designed to make programming more interactive

By Perfect Phanitchaleun and Nicolaus ReyasBautista

General Idea

01

Chess as Code Execution

In our language, each chess move represents an instruction. The sequence of moves creates a program, similar to how lines of code execute step-by-step.

02

Chessboard as Execution Visualization

The board is not memory itself, but a representation of how the program runs. Piece movement shows how data and logic change over time in a structured, turn-based system.

03

Pieces as Data Types

Different pieces and playstyles define data types (e.g., pawns for variables, aggressive play for strings, defensive play for integers).

04

Moves Encode Binary

By moving pieces across specific tiles (like ranks/files), we encode binary digits (0s and 1s), allowing us to assign values to integers and strings.

05

Turn-Based Execution

Alternating turns act like sequential execution. Each side contributes to building and modifying the program state step-by-step.

1. State-Based Execution

Program logic is determined by the **current board state**, not individual moves. Positions of pieces act like registers that define how the program behaves.

2. Mode Switching via Knight

The White knight selects the **data type / mode**:

- **Nb1** → **a3** → Integer mode
- **Nb1** → **c3** → String mode

This acts like declaring a variable type in traditional programming.

3. Variable Naming by Coordinates

Variables are named using **pawn positions** (e.g., p_h2). The board itself provides a natural naming system based on coordinates.

4. Rook as a Write Head (Binary Encoding)

The Black rook functions as a **write head**, similar to a tape machine. Moving along the 6th rank (**b6** → **h6**) encodes binary values that form integers or ASCII characters.

Example Program

1. W Nb1 a3
2. B Pa7 a5
3. W Ph2 h3
4. B Pa5 a4
5. W Ph3 h4
6. B Ra8 a6
7. W K a3 c4
8. B R a6 b6 # (1000000)
9. W K c4 a3
10. B R b6 c6 # (1100000)
11. W K a3 c4
12. B R c6 f6 # (1100100)
13. W K c4 a3
14. B R f6 a6
15. W K a3 b1
16. B R a6 a8

In Java:

int p_h2 = 100

Setup Phase (Moves 1–2)

W Nb1 → a3
→ *Instruction:* begin_int p_h2
The knight moving to a3 signals the start of an integer variable initialization.

B Pa7 → a5 → a4
→ *Instruction:* prepare_rook_access
Black clears a path so the rook can move freely and be used for encoding.

B Ra8 → a6
→ *Instruction:* enter_encoding_mode
The rook reaching the 6th rank activates binary encoding mode.

Encoding Phase (Moves 3–5)

B R a6 → b6 → c6 → f6
→ *Instruction:* write_bits(1100100)
Each rook position corresponds to binary digits.
The sequence of movements builds the binary value **1100100 = 100**.

W N a3 ↔ c4 (repeated)
→ *Instruction:* no_op / wait
These are "waiting moves." Since turns must alternate, White performs neutral moves to allow Black to continue encoding.

Finalize Phase (Moves 6–7)

B R f6 → a6 → a8
→ *Instruction:* end_encoding
Returning the rook resets the board and signals completion of binary input.

W N a3 → b1
→ *Instruction:* commit p_h2
The knight returning to its original square finalizes and stores the integer value.

Example Program

17. W Pd2 d3
18. B Ph7 h5
19. W Qd1 d2 # Initiate printing
20. B Ph5 h4
21. W Ph2 h3 # Printing variable P_h2
22. B Ph4 h3
23. W Qd2 d1 # Finalizing printing

Print Function (Concept) / Sample 2

→ *Instruction:* print(P_h2)

By moving the Queen and moving the pawn with the address of the variable we wanted to print

Ending the program Through Checkmate (Concept) / Sample 3

→ *Instruction:* exit()

White checkmated Black through the scholars check mate, ending the program.

24. W Pe2 e3
25. B Pf7 f6
26. W Bf1 c4
27. B Ng1 h6
28. W Qd1 h5
29. B Nh6 f7
30. W Qh5 f7 # Checkmate, ending the program